

PROJECT OVERVIEW

Mini ERP

An inventory and order management app for small and mid-sized trading businesses: purchasing, goods receipt, sales, and per-warehouse stock in one flow, plus a mobile app for managers.

Raihan Evanza · Fullstack & Mobile Developer · portfolio-raihanevanza.vercel.app

ROLE

Personal project · database design, backend, web, mobile, deployment

PLATFORMS

Web (desktop) · Android & iOS (manager-only app)

LINKS

[Live demo: raihan-evanza-erp.vercel.app](https://raihan-evanza-erp.vercel.app)

Go 1.22

Gin

GORM

SQL Server 2022

JWT

React 18

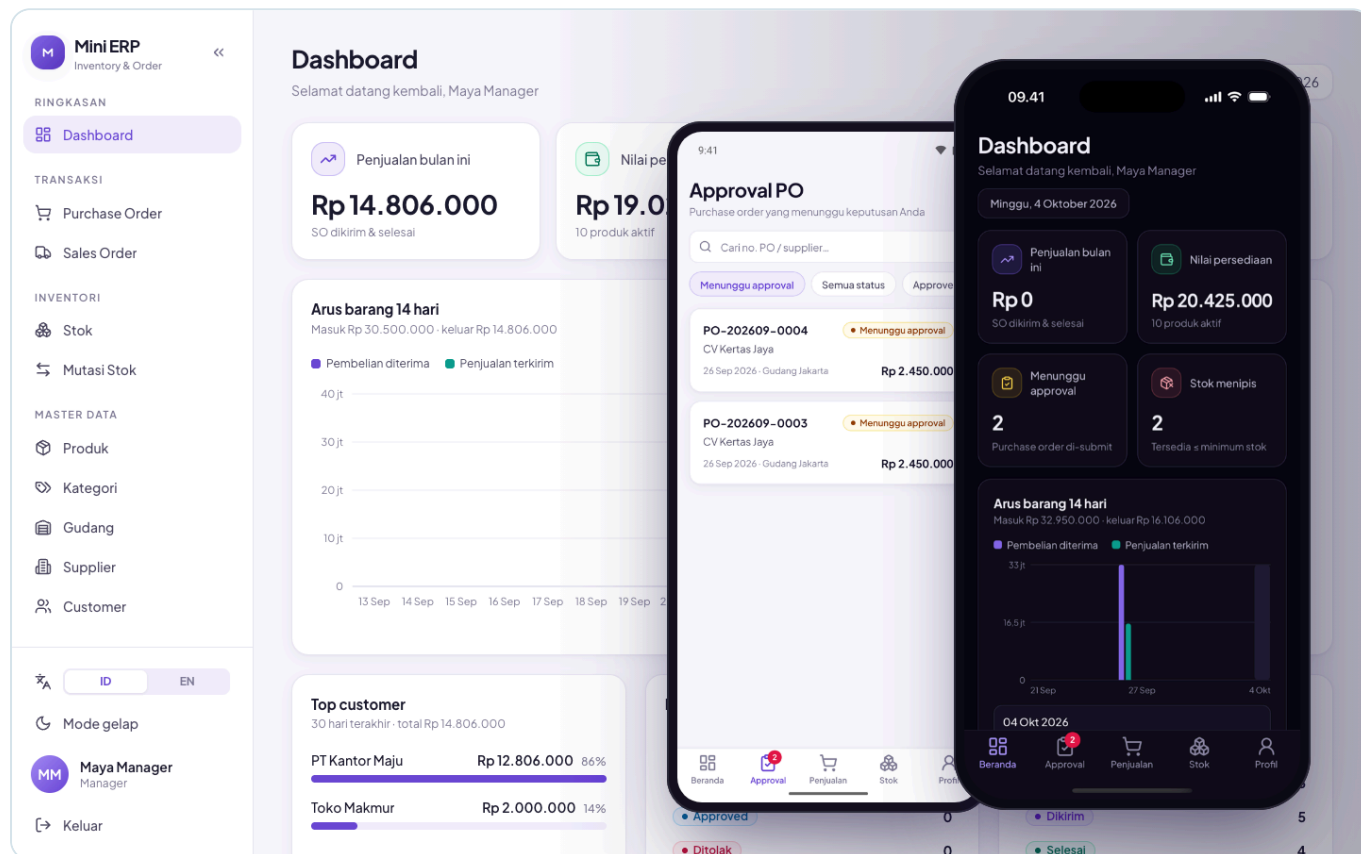
TypeScript

Vite

Redux Toolkit (RTK Query)

Tailwind CSS

React Native 0.87



OVERVIEW

Trading businesses often track stock in a spreadsheet that is separate from purchasing and sales. Stock figures drift from reality, goods already promised to a customer get sold again, and it is unclear who approved a purchase.

Mini ERP puts that flow in one place: a purchase order (PO) must be approved before goods are received, receipts can arrive in several batches, a sales order (SO) reserves stock as soon as it is confirmed, and every stock change is written to a per-warehouse movement ledger. Access is split by role (Admin, Manager, Warehouse, Sales).

The result is a Go backend on SQL Server, a React web app that runs without a server through a demo mode on Vercel, and a React Native app that lets managers watch the dashboard and decide on PO approvals from their phone.

KEY FEATURES

Purchase orders with approval

The flow is Draft → Submitted → Approved → Received. The person who created a PO cannot approve or reject it, and a rejection requires a reason.

Partial goods receipt

The warehouse records only what actually arrived; the rest stays open and the PO shows "Partially received". Each receipt gets its own GR number and can never exceed the ordered quantity.

Sales orders with stock reservation

When an SO is confirmed, stock for every item is reserved at once; if one item is short, the whole confirmation is rolled back with a message showing how much is available. Shipping reduces on-hand stock, and cancelling a confirmed SO releases its reservation.

Per-warehouse stock & ledger

Stock is kept per product per warehouse as on hand, reserved, and available (on hand – reserved), with a low-stock flag. The Stock Movements page lists every change with the balance after it and its source document (GR or SO).

Role-based access

Four roles with different permissions: Admin has full access, Manager approves POs and edits master data, Warehouse creates POs, receives goods, and ships SOs, and Sales manages SOs. Buttons a user may not use are hidden, and the backend still rejects the call with a 403.

KPI dashboard

Sales this month, inventory value, POs awaiting approval, and low-stock count, plus a 14-day goods flow chart, stock value per category, and top customers. The charts are hand-built SVG and can be switched to a table view.

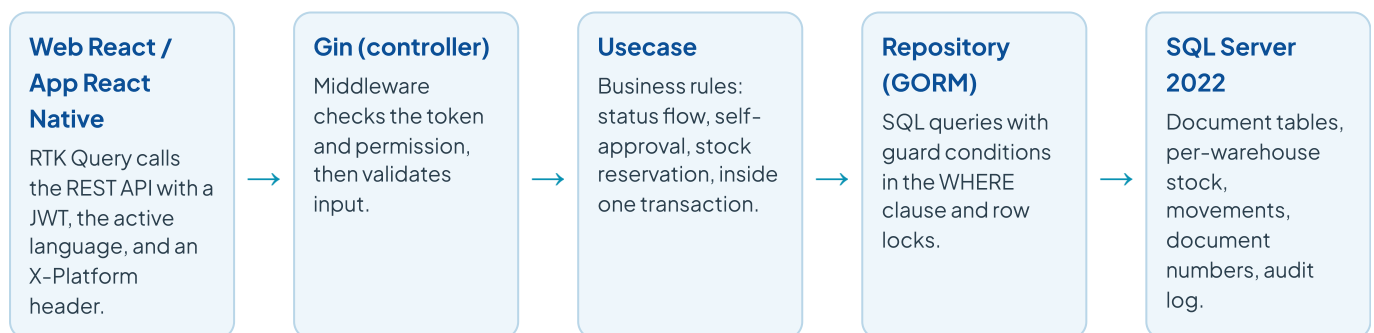
Two languages & dark mode

The interface is available in Indonesian and English, including backend error messages, which follow the Accept-Language header. The light/dark theme follows the system setting and can be switched manually.

Mobile app for managers

A React Native app (Android + iOS) for Managers and Admins: dashboard, a PO approval inbox with approve/reject, and monitoring of SOs and low stock. Other roles are turned away at login because they lack approval permission.

ARCHITECTURE & DATA FLOW



- The Go backend has three layers (controller → usecase → repository) so business rules can be unit-tested with GoMock without a database.
- A single transaction helper passes the transaction through the context, so every repository called within one action uses the same transaction automatically, including the audit log write.
- In the Vercel demo, everything from Gin to SQL Server is replaced by an in-browser Axios adapter that applies the same rules and keeps data in localStorage.

STACK BY LAYER

Backend	Go 1.22 · Gin · GORM · JWT (HS256) + bcrypt · Viper · go-i18n · Swaggo (Swagger) · Testify + GoMock
Database	SQL Server 2022 (Docker) · CHECK constraint · Filtered unique index (soft delete) · View vw_LowStock
Web	React 18 · TypeScript · Vite · Redux Toolkit (RTK Query) · React Router · react-hook-form + yup · Tailwind CSS · Vercel
Mobile	React Native 0.87 (New Architecture, Hermes) · React Navigation 7 · RTK Query · react-native-keychain · react-native-svg

TECHNICAL DECISIONS

1 Every stock change in one transaction

A goods receipt writes the GR, increments the received quantity, increases stock, and logs the movement in one transaction; if any step fails, all of it is rolled back. Stock updates use UPDLOCK/ROWLOCK with a "stock is sufficient" condition in the WHERE clause, and the stock table has CHECK constraints (reserved ≤ on hand, nothing negative) as the last safety net.

2 Status guards in the query, not check-then-write

Status changes use UPDATE ... WHERE Status IN (allowed source statuses). If two people press the button at the same time, only one succeeds and the other gets a 409, so a document is never processed twice.

3 Reserve on SO confirmation

Stock is deducted only when goods ship, but it is reserved as soon as the SO is confirmed. That way stock promised to one customer cannot be sold to another, and the SO form shows available stock before confirmation.

4 Role → permission mapping in one place

The permission list per role lives in a single map in the backend and is sent to the client through /auth/me. Web and mobile check permissions (e.g. po.approve) rather than role names, so changing access rules touches one file.

5 Serverless demo mode on Vercel

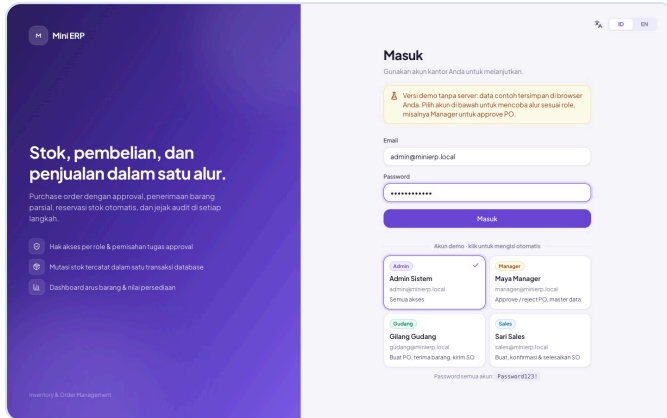
So the app can be tried without hosting Go and SQL Server, the demo build swaps the Axios adapter for an in-browser backend that applies the same rules: permissions, status flow, reservation, the self-approval ban, and bilingual errors. A 14-day history is regenerated relative to today, and the demo code is excluded from the normal build.

6 Mobile app for managers only

Daily data entry (creating POs/SOs, receiving goods) is easier on a large screen, while managers need to approve from anywhere. So the mobile app only covers monitoring and approve/reject, with buttons locked while a request is in flight to prevent duplicate decisions.

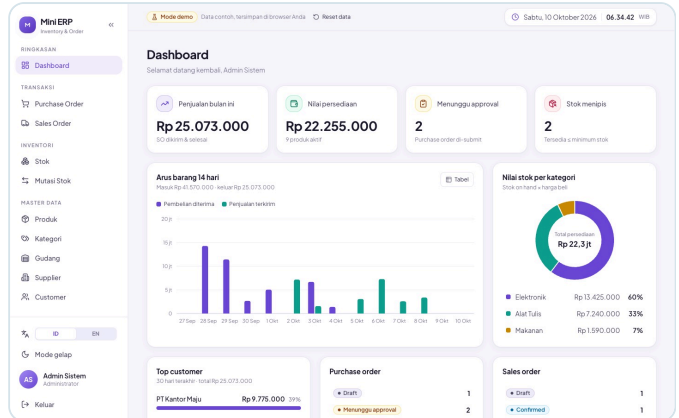
HOW TO USE IT

These steps follow the same order as the tutorial video.



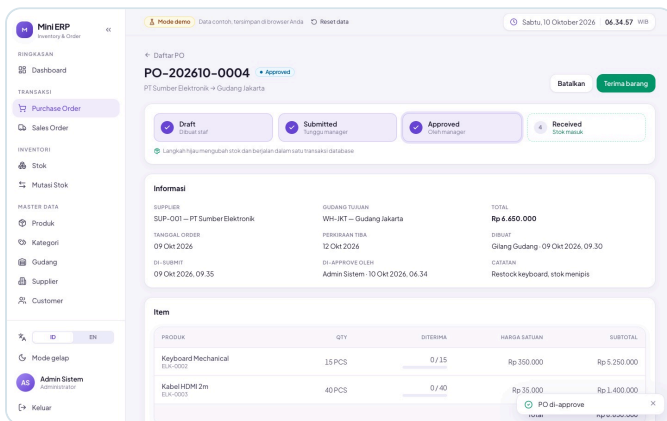
1 Sign in with a demo account

Open the demo, click one of the demo account cards (for example Admin Sistem) to fill in the email and password, then click Sign in. Every account uses the password Password123! and shows the menu its role allows.



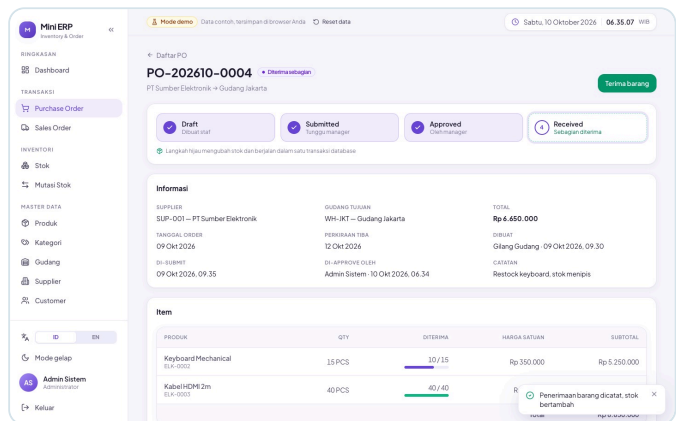
2 Read the dashboard

Four KPI cards at the top show sales this month, inventory value, POs awaiting approval, and low stock. Below them are the 14-day goods flow chart, stock value per category, and top customers.



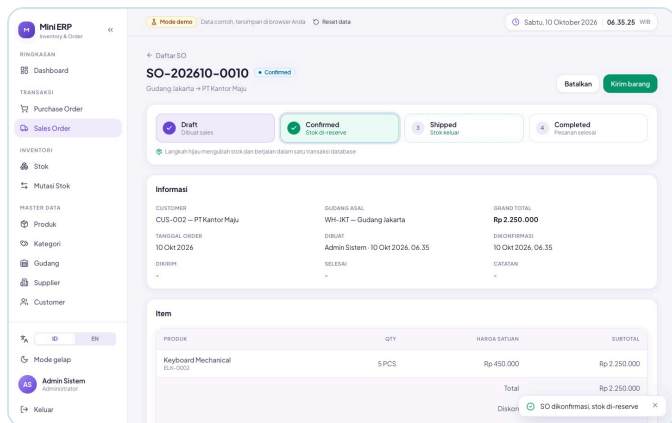
3 Approve a purchase order

Open Purchase Order, set the filter to "Awaiting approval", open the PO from PT Sumber Elektronik, and click Approve. This PO was created by the Warehouse account; its creator would not see the Approve button.



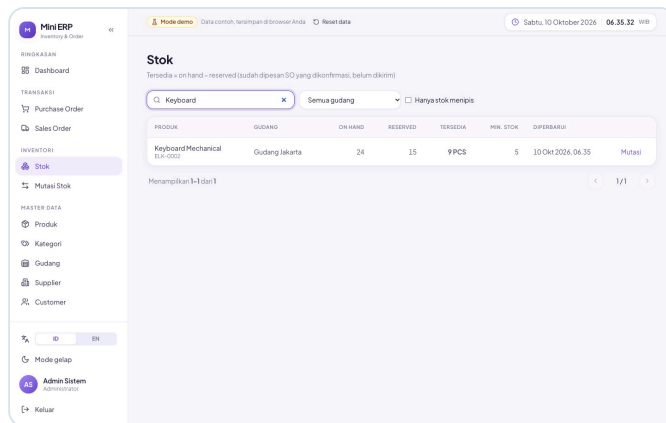
4 Receive goods partially

Click Receive goods, change Keyboard Mechanical to 10 of 15, and save. The PO status becomes "Partially received" and a GR number appears in the receipt history.



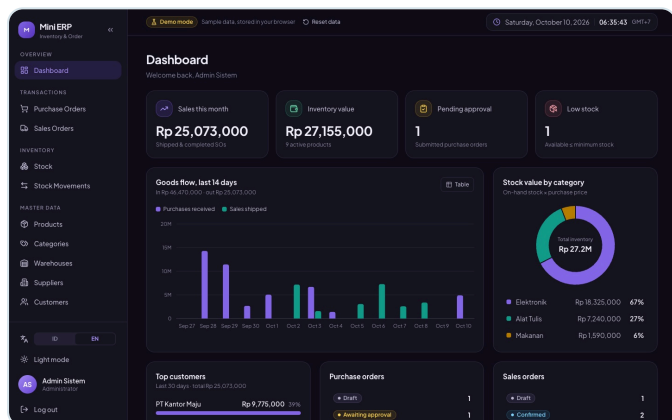
5 Create a sales order and reserve stock

Go to Sales Order → Create SO, pick a customer, Gudang Jakarta, and Keyboard Mechanical with qty 5; the form shows available stock right away. Save it as a draft, then click "Confirm & reserve stock".



6 Check stock and the movement ledger

On the Stock page, search "Keyboard" to see on hand, reserved, and available change after the last two steps. The Stock Movements page shows a "Purchase in" row of +10 with the balance after it and the GR number as its reference.



7 Switch theme and language

At the bottom of the sidebar, click Dark mode and pick EN to see the whole interface in English. The Reset data button in the header restores the demo data to its initial state.

LIMITATIONS & NOTES

- The live demo runs without a backend: data lives in each visitor's browser localStorage and the business rules are reimplemented in TypeScript rather than executed by the Go server.
- The mobile app needs a running Go backend (local or hosted) and has not been published to the Play Store/App Store; iOS signing is not set up yet.
- ROWVERSION columns exist in the schema but the API does not use them yet, so concurrent edits of the same PO/SO draft are still last-write-wins. Status and stock changes are already protected by WHERE guards.
- There is no refresh token yet (the session ends when the JWT expires), and the stock adjustment (stock count) tables have no endpoint or page yet.

WHAT'S NEXT

- Stock adjustments (stock counts) with approval, using the tables already in the schema.
- Push notifications to the manager app when a PO is submitted, plus refresh tokens and deep links to PO details.
- Hosting the Go backend and SQL Server so the web and mobile demos run against a real server.